

Lab 2 report

2026/5/2
Xue Zhengyang

1. Introduction

This report summarizes a series of hands-on experiments designed to establish a foundational understanding of Docker containerization technology. The practical exercises covered five core areas. First, publishing container ports via the Docker CLI. Second, sharing files between host and container using bind mounts. Third, persisting data across container lifecycles using Docker volumes. Fourth, building and orchestrating a multi-container application with Nginx, Node.js, and Redis using both imperative docker run commands and declarative Docker Compose. Fifth, overriding container defaults including networking configurations, resource limits, and entrypoint commands. Through these exercises, the theoretical abstraction of containers was translated into concrete operational skills.

2. Experimental Setup

The experiments were conducted on a Windows host machine running Docker Desktop. The primary interface for command execution was Windows PowerShell. Prerequisites included the installation of Docker Desktop with WSL2 integration enabled, as well as Git for repository cloning. Key images used during the experiments included docker/welcome-to-docker, httpd:2.4, postgres:18, redis, and custom-built images for a Node.js web application and an Nginx reverse proxy.

3. Experiment 1: Container Port Publishing with Docker CLI

Objective: Understand how to expose container services to the host machine through port mapping.

Procedure: A container was launched using the command `docker run -d -p 8080:80 docker/welcome-to-docker`. The syntax `HOST_PORT:CONTAINER_PORT` was analyzed, where port 8080 on the localhost was bound to port 80 inside the container.

Observation: The application became accessible via `http://localhost:8080`. The Docker Desktop Dashboard visually confirmed the published port mapping.

4. Experiment 2: Sharing Data with Bind Mounts

Objective: Implement real-time file synchronization between the host filesystem and a container using bind mounts.

Procedure: An initial `httpd:2.4` container was started and subsequently removed. A local directory named `public_html` was created, containing a custom `index.html` file with a Whalecome theme. A new container was then launched with a bind mount using the command `docker run -d --name my_site -p 8080:80 -v ${PWD}:/usr/local/apache2/htdocs/`

httpd:2.4. On Windows PowerShell, the absolute path `${PWD}` was required instead of the relative `./` notation commonly used in Unix-like shells.

Observation: The custom webpage rendered immediately at `http://localhost:8080`. When `index.html` was deleted from the host, the file simultaneously disappeared inside the container, which was verified via the Docker Desktop Dashboard Files tab. Apache then returned an Index of / listing. Recreating the file on the host restored the site instantly.

Key Insight: Bind mounts create a live, bidirectional link between host and container, making them ideal for development environments where code changes must be reflected immediately.

5. Experiment 3: Persisting Data with Docker Volumes

Objective: Demonstrate data persistence independent of container lifecycle using named volumes.

Procedure: A PostgreSQL container was initialized with a named volume using the command `docker run --name=db -e POSTGRES_PASSWORD=secret -d -v postgres_data:/var/lib/postgresql/data postgres:18`. A table named `tasks` was created, and two records were inserted. The container was then stopped and removed. A new container named `new-db` was started, attaching the same `postgres_data` volume, but omitting the `POSTGRES_PASSWORD` environment variable.

Observation: The data remained intact. Querying the new container with `SELECT * FROM tasks;` returned the previously inserted records. The volume was later inspected and removed using `docker volume rm postgres_data` and `docker volume prune`.

Key Insight: Unlike bind mounts, volumes are managed by Docker and are decoupled from the host filesystem structure. They are the preferred mechanism for database persistence and stateful applications.

6. Experiment 4: Multi-Container Orchestration with Docker Run

Objective: Manually wire a multi-service application consisting of a Redis database, two Node.js backend instances, and an Nginx reverse proxy.

Procedure: After cloning the `nginx-node-redis` sample repository, custom images were built for the web and nginx services. A dedicated bridge network named `sample-app` was created to facilitate DNS-based service discovery. Containers were launched sequentially. First, the redis container with a network alias. Second, `web1` and `web2` with hostnames and aliases. Third, the nginx container exposing port 80 and routing traffic to the backend containers.

Observation: Accessing `http://localhost` showed a visit counter that incremented on each refresh. The hostname alternated between `web1` and `web2`, confirming that Nginx was distributing requests in a round-robin fashion. Both backends shared the same Redis instance, ensuring a global, consistent counter state.

Challenge: Port 80 was initially occupied by a pre-existing XAMPP Apache service, causing a conflict. Stopping the XAMPP service resolved the issue.

7. Experiment 5: Streamlining Deployment with Docker Compose

Objective: Replace manual, imperative container orchestration with a declarative compose.yml configuration.

Procedure: Navigating to the project root, the existing compose.yml file was used to define the entire stack. The application was launched with a single command: `docker compose up -d --build`.

Observation: A single command created the default network and started all four services. The behavior was identical to the manual setup, but the operational complexity was drastically reduced. Cleanup was similarly simplified using `docker compose down`.

Key Insight: Docker Compose transforms multi-container deployment from an error-prone sequence of commands into a version-controlled, reproducible infrastructure specification. This aligns with Infrastructure as Code principles.

8. Experiment 6: Overriding Container Defaults and Resource Management

Objective: Explore advanced docker run flags for network isolation, resource constraints, and command overrides.

Procedure: Several independent tests were conducted. First, two PostgreSQL containers were run simultaneously on host ports 5432 and 5433, both mapping internally to container port 5432. Second, a user-defined bridge network named mynetwork was created. Unlike the default bridge network, custom networks provide automatic DNS resolution by container name and better isolation. Third, a container was constrained using `--memory="512m"` and `--cpus=".5"` to prevent resource starvation on shared systems. Fourth, both docker run and compose.yml were used to override the default PostgreSQL startup behavior, specifying custom host and port parameters.

Observation: Containers on the custom network could resolve each other by name rather than IP address. Resource-limited containers operated within their constraints without affecting host stability.

9. Challenges and Troubleshooting

Several challenges were encountered during the experiments.

The Git clone operation initially failed with an RPC error and connection reset. This was caused by network instability or firewall interference. The issue was resolved by using a shallow clone with `git clone --depth 1` or by downloading the ZIP archive directly from GitHub.

When attempting to create the public_html directory, a permission denied error occurred because the command was executed in `C:\WINDOWS\system32`, which is a protected system directory. The solution was to navigate to the user home directory using `cd ~` before creating files.

A path error occurred when using the `cd` command with a username containing a space. The path `C:\Users\Xue Zhengyang` was parsed incorrectly by PowerShell. The solution was to use quoted paths such as `cd "C:\Users\Xue Zhengyang"`.

During the multi-container experiment, the browser displayed the XAMPP welcome page instead of the Docker application. This happened because the XAMPP Apache service was already running and occupying port 80. The conflict was resolved by stopping the Apache service through the XAMPP Control Panel.

Finally, PowerShell has a built-in alias that maps `curl` to `Invoke-WebRequest`, which behaves differently from the standard `curl` utility. This caused URI parsing errors. The solution was to use `curl.exe` with the explicit protocol prefix <http://>.

10. Key Takeaways and Reflections

The Ephemeral Nature of Containers vs. The Necessity of Persistence

The most profound realization from this series was the distinction between a container's ephemeral filesystem and persistent storage mechanisms. Without explicit volumes or bind mounts, container data vanishes upon deletion. This makes Docker volumes indispensable for stateful services like databases, while bind mounts excel in development workflows requiring rapid iteration.

Imperative vs. Declarative Orchestration

Running the Nginx-Node-Redis stack manually with six separate commands including `docker network create`, four `docker run` commands, and verification steps was tedious and brittle. A single typo in a network alias or port mapping would break the entire stack. Docker Compose eliminated this fragility by centralizing configuration. This experience solidified my appreciation for declarative tooling in modern DevOps practices.

Networking as a First-Class Concern

The default bridge network's lack of DNS resolution was a subtle but critical discovery. In real-world microservices architectures, services must discover each other dynamically. Custom bridge networks provide this capability natively, reinforcing the principle that container networking should never be an afterthought.

Cross-Platform Nuances

Working in Windows PowerShell revealed friction points that are invisible in Unix environments. These included the need for absolute paths in bind mounts, the space-character issue in file paths, and the `curl` alias conflict. These challenges underscored that containerization abstracts the runtime environment, but the host environment still matters during development and debugging.

Resource Governance

The resource-limiting exercise using `--memory` and `--cpus` served as a reminder that containers, while isolated, share the host kernel. On multi-tenant or production systems, unconstrained containers pose a stability risk. Proactive resource management is not merely an optimization but an operational requirement.

The Power of Layered Abstraction

From a single `docker run -p` command to a multi-service `compose.yml`, the experiments traced a clear trajectory of increasing abstraction. Yet each layer builds upon the previous one. Understanding `docker run` is a prerequisite to debugging Compose failures. This reinforces the value of mastering fundamentals before adopting higher-level tools.

11. Conclusion

This comprehensive Docker laboratory provided end-to-end exposure to container lifecycle management, storage strategies, networking, and orchestration. The progression from isolated containers to a load-balanced, multi-service architecture, first managed manually and then via Docker Compose, mirrors the evolution of container usage in industry settings. The hands-on troubleshooting of port conflicts, path syntax issues, and network behavior transformed abstract documentation into operational intuition. Moving forward, these skills form the foundation for more advanced topics, including container security, CI/CD pipeline integration, and Kubernetes orchestration.